

UNIVERSIDAD DEL VALLE DE GUATEMALA

PROGRAMACION ORIENTADA A OBJETOS

DIEGO ISAIAS MARROQUIN MOSCOSO

EJERCICIO #4

Análisis

Análisis del sistema de alquiler de vehículos RentaMovil:

1. ¿Qué clases son necesarias para representar el problema planteado?

Son necesarias seis clases: Vehiculo, Automovil, Motocicleta, CamionetaCarga, RentaMovil y Main. Vehiculo reúne los datos y operaciones comunes. Las tres subclases agregan sus características y reglas de cobro. RentaMovil administra la flota y los ingresos. Main muestra el menú, recibe los datos y coordina la interacción con el usuario.

2. ¿Qué propiedades y métodos tendrá cada clase?

Vehiculo conserva placa, marca, modelo, tarifaD y disponibilidad; permite consultar sus datos, calcular la tarifa, alquilar, devolver y generar su descripción. Automovil agrega pasajeros y automatico; Motocicleta, cilindraje; CamionetaCarga, capacidadToneladas. Cada subclase redefine calcularTarifa() y toString().

RentaMovil conserva vehiculos e ingresosTotales. Permite registrar y buscar vehículos, mostrar la flota, cotizar y confirmar alquileres, registrar devoluciones, contar vehículos por estado y mostrar el reporte general.

Main contiene main(), crea RentaMovil y Scanner, carga seis vehículos disponibles y mantiene el menú de siete opciones. Sus datos de trabajo son variables locales.

3. ¿Qué tipos de datos y resultados utilizará el sistema?

Placa, marca y modelo son String; tarifaD, capacidadToneladas e ingresosTotales son double.

Pasajeros, cilindraje y días son int. Disponibilidad y automatico son boolean. Los importes se muestran con dos decimales.

La flota es ArrayList<Vehiculo>. Las búsquedas retornan Vehiculo o null; las operaciones de registro, alquiler y devolución retornan boolean. Las cotizaciones retornan double; los conteos, int; las descripciones, String. Los métodos que solo imprimen información retornan void.

main(String[] args) es public static void. Los constructores inicializan objetos y no declaran tipo de retorno. Las firmas completas se especifican en la descripción de cada clase.

4. ¿Qué información debe conservar el sistema?

Debe conservar los datos comunes y particulares de cada vehículo, su disponibilidad y el total acumulado de alquileres confirmados. La flota almacena referencias a objetos de las tres subclases mediante Vehiculo. Los datos permanecen únicamente durante la ejecución; no se requieren fechas, clientes, reservas, multas, reembolsos ni archivos de datos.

5. ¿Cómo se organizará la colección dinámica?

RentaMovil declara vehiculos como `ArrayList<Vehiculo>` y lo crea con `new ArrayList<Vehiculo>()` en su constructor. La misma colección admite automóviles, motocicletas y camionetas gracias a la herencia. `registrarVehiculo()` agrega objetos después de comprobar que su placa no esté registrada.

6. ¿Cuáles son los modificadores de visibilidad?

Los cinco atributos de `Vehiculo` son `protected`, tal como están declarados en el código. Los atributos particulares de las subclases y los de `RentaMovil` son `private`. `descripcionCosto` es `private static final`. Los constructores y métodos del modelo son `public`; `main()` además es `static`. `Main` accede al modelo mediante métodos.

7. ¿Qué parámetros necesitan las operaciones?

- `Vehiculo` recibe `placa`, `marca` y `modelo` como `String`, `tarifaD` como `double` y `disponibilidad` como `boolean`.
- `Automovil` recibe `pasajeros`: `int` y `automatico`: `boolean`, seguidos de los cinco parámetros de `Vehiculo`.
- `Motocicleta` recibe `cilindraje`: `int` y los cinco parámetros comunes.
- `CamionetaCarga` recibe `capacidad`: `double` y los cinco parámetros comunes.
- `registrarVehiculo(vehiculo: Vehiculo)` recibe el objeto que se incorporará a la flota.
- `buscarVehiculo(placa: String)` y `registrarDevolucion(placa: String)` identifican el vehículo por su placa.
- `cotizarAlquiler(placa: String, dias: int)` y `confirmarAlquiler(placa: String, dias: int)` reciben placa y duración.

- `calcularTarifa(dias: int)` recibe la duración del alquiler en días enteros positivos.
- `RentaMovil()`, getters, conteos, `mostrarFlota()`, `mostrarReporte()`, `alquilar()`, `devolver()` y `toString()` no reciben parámetros.
- `main(args: String[])` recibe el arreglo de argumentos de Java.

8. ¿Cómo se asignan los valores iniciales?

Los constructores reciben los datos del vehículo; las subclases envían los comunes a `super(...)`.

`RentaMovil` crea una lista vacía e inicia `ingresosTotales` en 0. `Main` registra dos vehículos de cada categoría, todos con `disponibilidad=true` y placas diferentes. Incluye un automóvil automático, uno manual y motocicletas de 150 y 300 cc, para demostrar ambos cobros.

9. ¿Cómo se identifica un vehículo y se evitan duplicados?

`buscarVehiculo(placa)` recorre la colección y compara con `getPlaca().equalsIgnoreCase(placa)`.

Retorna el objeto encontrado o null. `registrarVehiculo()` rechaza la incorporación si la búsqueda ya devuelve un objeto. `Main` rechaza las placas vacías antes de construir el vehículo.

10. ¿Cuáles son las reglas de cobro?

La base es $\text{tarifaD} \times \text{días}$. `Automovil` suma $Q50.00 \times \text{días}$ cuando es automático; el manual no tiene recargo. `Motocicleta` suma $Q75.00$ una sola vez si `cilindraje > 250`. `CamionetaCarga` suma $Q100.00 \times \text{capacidadToneladas} \times \text{días}$, sin depender de la carga transportada. Con tarifa de $Q200.00$, capacidad de 1.5 t y tres días, el total es $Q1,050.00$.

11. ¿Cómo se diferencian cotizar, confirmar y cancelar?

Cotizar obtiene el precio sin cambiar disponibilidad ni ingresos, incluso si el vehículo está ocupado.

Main muestra el importe y solicita SI o NO. Solo con SI llama a confirmarAlquiler(), que exige placa existente, días positivos y disponibilidad. Si alquilar() retorna true, acumula el precio completo. Al responder NO no se ejecuta esa operación y el estado permanece igual.

12. ¿Qué ocurre al devolver un vehículo y consultar el reporte?

registrarDevolucion() localiza el vehículo y llama a devolver(). Solo cambia su estado si estaba alquilado; no modifica ingresosTotales. mostrarReporte() cuenta disponibles y alquilados por categoría, muestra la cantidad de cada tipo y el total general, además del dinero acumulado por confirmaciones exitosas.

13. ¿Cómo se aplican la herencia y el polimorfismo?

Las subclases heredan los datos comunes, getters y operaciones de disponibilidad de Vehiculo. Cada una redefine calcularTarifa() y reutiliza la base mediante super.calcularTarifa(dias). RentaMovil invoca ese método a través de una referencia Vehiculo; Java selecciona la versión correspondiente al objeto real. Así, la gestión del alquiler no repite las fórmulas de cada categoría.

14. ¿Qué condiciones invalidan una operación?

No se permiten placas vacías o repetidas, tarifas o características numéricas menores o iguales a cero, ni días que no sean enteros positivos. Tampoco se permite alquilar un vehículo ocupado o devolver uno disponible. Una placa inexistente debe producir un mensaje sin alterar datos. Las entradas de formato incorrecto deben permitir reintentar.

15. ¿Dónde se realizan las validaciones y cómo se controlan los errores?

Main valida los datos que ingresa el usuario y utiliza try-catch, incluyendo InputMismatchException al leer números. RentaMovil comprueba duplicados, existencia, días y disponibilidad en sus operaciones. Los constructores actuales asignan los valores recibidos; por ello, la validación del registro depende también de Main. Las cotizaciones inválidas retornan -1 y las operaciones rechazadas retornan false.

16. ¿Qué tendría que cambiar si se agrega otra categoría?

Se crearía otra subclase de Vehiculo con sus atributos, constructor y cálculo de tarifa. La colección y las operaciones generales seguirían funcionando mediante Vehiculo. Habría que ampliar Main para pedir los datos de la categoría y modificar mostrarReporte(), que actualmente distingue los tres tipos con instanceof. El cálculo y la confirmación del alquiler no necesitarían otra fórmula dentro de RentaMovil.

Requisitos funcionales del sistema:

- Registrar automóviles, motocicletas y camionetas con placa única, marca, modelo, tarifa diaria y disponibilidad inicial.
- Conservar pasajeros y transmisión del automóvil, cilindraje de la motocicleta y capacidad decimal de la camioneta.
- Validar placas no vacías ni repetidas y valores numéricos mayores que cero.
- Consultar la flota y localizar cada vehículo por placa.
- Cotizar con días enteros positivos, mostrando características, disponibilidad y total, sin modificar el estado.
- Aplicar la tarifa diaria y los recargos específicos de cada categoría.
- Verificar disponibilidad y mostrar el monto antes de confirmar un alquiler.
- Cobrar y acumular el monto completo solo al confirmar un alquiler válido.
- Conservar disponibilidad e ingresos cuando el usuario cancela o una operación se rechaza.
- Registrar la devolución de un vehículo alquilado sin nuevos cobros ni eliminación de ingresos.
- Mostrar cantidades por categoría y estado, el total de vehículos y los ingresos acumulados.
- Mantener un menú de siete opciones hasta que el usuario seleccione salir.
- Manejar entradas incorrectas sin terminar inesperadamente el programa.
- Iniciar con dos vehículos de cada categoría, placas distintas, todos disponibles e ingresos en Q0.00.
- Presentar los importes con dos decimales y conservar los datos solo durante la ejecución.
- Permitir la incorporación futura de categorías reutilizando los datos y operaciones comunes de Vehículo.

Descripción de las clases

Notación: + public; - private; # protected. Se describen los miembros declarados; los heredados conservan el comportamiento de Vehiculo. static indica un miembro de clase y final, un valor que no se reasigna.

Vehiculo

Propósito de la clase:

Representa los datos y comportamientos comunes de un vehículo. Es una clase concreta y sirve como superclase de las tres categorías. Mantiene la disponibilidad y calcula el costo base del alquiler.

Propósito de los atributos:

placa: String. Identifica el vehículo y permite buscarlo y evitar duplicados.

marca: String. Conserva el fabricante.

modelo: String. Conserva la identificación del modelo.

tarifaD: double. Guarda el precio base por día.

disponibilidad: boolean. true significa disponible; false, alquilado.

Propósito de los métodos:

+ Vehiculo(placa: String, marca: String, modelo: String, tarifaD: double, disponibilidad: boolean).

Asigna los cinco valores recibidos. El registro desde Main utiliza disponibilidad=true.

+ getPlaca(): String. Retorna la placa.

+ getMarca(): String. Retorna la marca.

+ getModelo(): String. Retorna el modelo.

- + getDisponibilidad(): boolean. Retorna el estado actual.
- + calcularTarifa(dias: int): double. Retorna $\text{tarifaD} \times \text{dias}$ sin modificar el vehículo.
- + alquilar(): boolean. Si está disponible, cambia disponibilidad a false y retorna true; si ya está ocupado, retorna false.
- + devolver(): boolean. Si está alquilado, cambia disponibilidad a true y retorna true; si ya estaba disponible, retorna false.
- + toString(): String. Retorna marca, modelo, placa, disponibilidad y tarifa diaria con dos decimales.

Automovil

Propósito de la clase:

Especializa Vehiculo para automóviles. Conserva capacidad de pasajeros y transmisión, y aplica el recargo diario solo si la transmisión es automática.

Propósito de los atributos:

- pasajeros: int. Cantidad de pasajeros admitidos.
- automatico: boolean. Indica si aplica el recargo por transmisión automática.
- descripcionCosto: String {static final}. Texto fijo que describe el recargo de Q50.00 por día.

Propósito de los métodos:

- + Automovil(pasajeros: int, automatico: boolean, placa: String, marca: String, modelo: String, tarifaD: double, disponibilidad: boolean). Inicializa los datos comunes con super(...) y asigna pasajeros y automatico.
- + pasajeros(): int. Retorna la cantidad de pasajeros.
- + automatico(): boolean. Retorna el tipo de transmisión.

+ calcularTarifa(dias: int): double. Redefine el método heredado; suma $Q50.00 \times \text{dias}$ al costo base si automatico es true.

+ toString(): String. Amplía la descripción heredada con pasajeros, transmisión y descripción del recargo.

Motocicleta

Propósito de la clase:

Especializa Vehiculo para motocicletas. Utiliza el cilindraje para decidir si se agrega un recargo único al alquiler.

Propósito de los atributos:

- cilindraje: int. Cilindrada del motor en centímetros cúbicos.

- descripcionCosto: String {static final}. Texto fijo que describe el recargo de Q75.00 cuando el cilindraje supera 250 cc.

Propósito de los métodos:

+ Motocicleta(cilindraje: int, placa: String, marca: String, modelo: String, tarifaD: double, disponibilidad: boolean). Inicializa los datos comunes con super(...) y asigna cilindraje.

+ getCilindraje(): int. Retorna la cilindrada.

+ calcularTarifa(dias: int): double. Redefine el cálculo y suma Q75.00 al costo base si cilindraje > 250, una sola vez por alquiler.

+ toString(): String. Amplía la descripción de Vehiculo con cilindraje y descripción del recargo.

CamionetaCarga

Propósito de la clase:

Especializa Vehiculo para camionetas de carga. Calcula el recargo según la capacidad máxima y la duración, aunque se transporte menos carga.

Propósito de los atributos:

- capacidadToneladas: double. Capacidad máxima de carga; permite valores decimales.
- descripcionCosto: String {static final}. Texto fijo que describe el recargo de Q100.00 por tonelada de capacidad por día.

Propósito de los métodos:

- + CamionetaCarga(capacidad: double, placa: String, marca: String, modelo: String, tarifaD: double, disponibilidad: boolean). Inicializa los datos comunes con super(...) y asigna capacidad a capacidadToneladas.
- + getCapacidadToneladas(): double. Retorna la capacidad máxima.
- + calcularTarifa(dias: int): double. Redefine el cálculo y suma $Q100.00 \times \text{capacidadToneladas} \times \text{dias}$ al costo base.
- + toString(): String. Amplía la descripción heredada con capacidad y descripción del recargo.

RentaMovil

Propósito de la clase:

Administra una colección de vehículos, coordina los alquileres y conserva los ingresos confirmados. Delega el cálculo del precio a cada vehículo y evita duplicar las reglas de las subclases.

Propósito de los atributos:

- vehiculos: ArrayList<Vehiculo>. Almacena los vehículos registrados de cualquier categoría.
- ingresosTotales: double. Acumula únicamente los importes de alquileres confirmados.

Propósito de los métodos:

- + RentaMovil(). Crea vehiculos como ArrayList vacío e inicializa ingresosTotales en 0.
- + getIngresosTotales(): double. Retorna el dinero acumulado.
- + registrarVehiculo(vehiculo: Vehiculo): boolean. Rechaza una placa repetida; en otro caso agrega el objeto y retorna true. Los demás datos se validan en Main.
- + buscarVehiculo(placa: String): Vehiculo. Busca sin distinguir mayúsculas y minúsculas; retorna el vehículo o null.
- + mostrarFlota(): void. Imprime los vehículos mediante sus versiones de toString(); informa si la flota está vacía.
- + cotizarAlquiler(placa: String, dias: int): double. Retorna calcularTarifa(dias) si la placa existe y los días son positivos; en otro caso retorna -1. No exige disponibilidad ni cambia ingresos.
- + confirmarAlquiler(placa: String, dias: int): boolean. Verifica existencia, días positivos y disponibilidad. Calcula el costo, llama a alquilar() y suma el importe si el alquiler se realiza. Retorna false ante un rechazo.
- + registrarDevolucion(placa: String): boolean. Retorna false si no existe la placa; de otro modo retorna el resultado de devolver(). No modifica ingresos.
- + contarVehiculos(): int. Retorna el tamaño de la lista.
- + contarDisponibles(): int. Cuenta objetos con disponibilidad=true.
- + contarAlquilados(): int. Cuenta objetos con disponibilidad=false.

+ mostrarReporte(): void. Cuenta los vehículos por categoría y disponibilidad mediante instanceof; imprime cantidades, total general e ingresos con dos decimales.

Main

+ main(args: String[]): void {static}

Propósito de la clase:

Contiene el punto de entrada y la interacción por consola. Crea RentaMovil y Scanner, carga dos vehículos de cada categoría y ofrece las opciones registrar, consultar flota, cotizar, confirmar alquiler, registrar devolución, mostrar reporte y salir.

Propósito de los atributos:

No declara atributos de instancia. scanner, rentaMovil, programaAbierto, validar, opcion, flotaInicial y los datos solicitados se manejan como variables locales dentro de main().

Propósito de los métodos:

main(args) mantiene activo el menú mediante while hasta seleccionar 7. Lee datos, controla errores de formato con try-catch, llama a las operaciones de RentaMovil y muestra sus resultados. Antes de confirmar un alquiler presenta el total y solicita SI o NO. No se declara un constructor explícito en Main.

Relaciones entre las clases:

- Automovil, Motocicleta y CamionetaCarga heredan directamente de Vehiculo. En UML, la línea continua con triángulo vacío apunta a Vehiculo.
- RentaMovil mantiene una asociación navegable hacia 0..* objetos Vehiculo. Recibe los objetos desde Main; no necesita crear cada vehículo ni controlar su existencia fuera de la lista.

- Main depende de RentaMovil para gestionar operaciones, usa Vehiculo como tipo común y crea objetos de las tres subclases.
- RentaMovil usa instanceof en mostrarReporte() para distinguir categorías; por ello también depende de las tres subclases en esa operación.

Colaboración para completar un alquiler

Main recibe placa y días, solicita la cotización a RentaMovil y muestra el monto. RentaMovil busca el Vehiculo y delega calcularTarifa() a su tipo concreto. Si el usuario acepta, Main solicita confirmarAlquiler(); RentaMovil verifica las condiciones, llama a alquilar() y acumula el precio. Si se cancela o se rechaza la operación, no se registra ingreso.

Evidencia de las pruebas realizadas

Se compilaron los seis archivos Java y se ejecutaron 30 comprobaciones directas del modelo, además de seis recorridos de Main mediante entradas de consola. Se resumen los resultados esperados y obtenidos. Los datos de cálculo del modelo son independientes de la flota de demostración de Main.

1. Cálculos de alquiler

Para tres días: automóvil automático con tarifa Q200.00, esperado Q750.00 y obtenido Q750.00; automóvil manual con la misma tarifa, esperado Q600.00 y obtenido Q600.00. Motocicleta de 250 cc con tarifa Q100.00, esperado Q300.00 y obtenido Q300.00; con 251 cc, esperado Q375.00 y obtenido Q375.00. Camioneta de 1.5 t y tarifa Q200.00, esperado Q1,050.00 y obtenido Q1,050.00.

2. Cotización sin cambios de estado

Después de cotizar los cinco casos, se esperaban ingresos Q0.00 y el automóvil disponible. Se obtuvieron Q0.00 y disponibilidad=true. También se cotizó un automóvil ya alquilado: se obtuvo Q500.00 por dos días sin cambiar sus ingresos acumulados.

3. Confirmación y doble alquiler

Al confirmar el automóvil automático por tres días se esperaba true, disponibilidad=false e ingresos Q750.00. Se obtuvieron esos tres valores. Al intentar alquilarlo otra vez se esperaba false y conservar Q750.00; esos fueron los resultados obtenidos.

4. Devolución y rechazos

Al devolver el automóvil se esperaba true, disponibilidad=true e ingresos Q750.00; se obtuvieron esos resultados. La segunda devolución, el alquiler y devolución de una placa inexistente y el alquiler de cero días retornaron false. Cotizar una placa inexistente o días negativos retornó -1. Los ingresos finales permanecieron en Q750.00.

5. Placas duplicadas y conteos

Después de registrar seis vehículos, se intentó agregar la placa a1 cuando ya existía A1. Se esperaba false y conservar seis vehículos; se obtuvieron esos resultados. Después de devolver el automóvil, los conteos esperados y obtenidos fueron seis disponibles y cero alquilados.

6. Cancelación desde el menú

En una ejecución nueva se eligió 4, placa PO8175X, tres días y NO. Se esperaba mostrar Q3,150.00 sin alquilar ni cobrar. Se obtuvo el mensaje de cancelación; el reporte mostró seis vehículos, ninguno alquilado e ingresos Q0.00.

7. Cotización y devolución desde el menú

En otra ejecución, cotizar PO8175X por tres días mostró Q3,150.00 y el reporte mantuvo Q0.00, como se esperaba. En una ejecución independiente se confirmó ese alquiler y luego se devolvió: se esperaban ingresos Q3,150.00 y cero alquilados; el reporte mostró ambos resultados.

8. Entradas incorrectas

Se ingresó abc como opción del menú: se esperaba rechazo y continuidad, y se obtuvo el aviso de número entero y salida normal posterior. Una placa vacía produjo un aviso y permitió cancelar. Al introducir 0 y abc como días, se solicitaron valores válidos; después de ingresar 2, se obtuvo Q2,100.00 y el programa continuó.

Observaciones sobre la implementación actual

La opción de cotización muestra marca, modelo, placa y total, pero debe incorporar la disponibilidad y las características particulares para cumplir completamente el enunciado. Además, la opción de confirmar solicita aceptación antes de comprobar la disponibilidad; RentaMovil impide el doble alquiler, pero conviene informar que está ocupado antes de pedir confirmación. Estas observaciones no afectan los resultados de cálculo y conservación de ingresos comprobados.

Las validaciones de placa vacía y valores positivos del registro están en Main; los constructores y registrarVehiculo() no las repiten todas. Por ello, si se incorporan otras formas de registrar vehículos, estas deben respetar las mismas condiciones. Main también limita pasajeros a 16 y rechaza espacios en marca y placa, restricciones adicionales al enunciado.